Week 18
RLHF & PPO for Language Models

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives **4**

1 Motivation & intuition **5**
1.1 Why classical RL is the wrong tool, naively . . . 5
1.2 The four-model picture . . . 5

2 Mathematical formulation **7**
2.1 Stage 0: the supervised fine-tuning baseline . . . 7
2.2 Stage 1: the Bradley–Terry reward model . . . 7
2.3 Stage 2: the KL-regularised RL objective . . . 8
2.4 Stage 3: PPO at the token level . . . 8
2.5 The full loss . . . 9
2.6 The adaptive KL controller . . . 9
2.7 A picture of token-level credit assignment . . . 10
2.8 An empirical illustration of the BT loss . . . 10

3 Algorithms **11**
3.1 The three-stage pipeline . . . 11
3.2 Worked numerical example . . . 11

4 Implementation walkthrough **12**

5 Empirical results **14**

6 Theoretical deepening **16**
6.1 Policy-gradient identity for autoregressive LMs . . . 16
6.2 Why a value head reduces variance . . . 16
6.3 Equivalence between the per-token KL and the sequence KL . . . 16
6.4 The KL controller as a PI loop in log-space . . . 17
6.5 Common questions, answered . . . 17
6.6 A second worked numerical example: one PPO step end-to-end . . . 17

7 Safety angle **19**

8 Matching application **20**

9 Exercises **21**

10 Notation, conventions, and detailed pseudocode **22**
10.1 Notation cheat sheet . . . 22
10.2 Detailed PPO rollout pseudocode . . . 23
10.3 Detailed PPO update pseudocode . . . 23
10.4 Memory accounting on a 24 GB GPU . . . 23
10.5 Hyper-parameter recipe summary . . . 24

11 Historical context and the road to GRPO **25**
11.1 From REINFORCE to InstructGPT, in five papers . . . 25

11.2 Why GRPO replaced PPO for reasoning RL 25
11.3 What was wrong with classical RLHF, exactly? 25

12 Further reading **26**

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Derive the *Bradley–Terry* pairwise preference model and its cross-entropy loss from first principles.
2. State the *KL-regularised RL objective* for language models and explain why each term exists.
3. Derive the PPO clipped surrogate at the *token level*, including Generalised Advantage Estimation (GAE), the value head, and the adaptive KL controller of Schulman et al. 2017.
4. Explain length bias, reward hacking, and reward-model overoptimisation, and pair each with a concrete mitigation.
5. Implement the full three-stage RLHF pipeline (SFT $\rightarrow$ RM $\rightarrow$ PPO) from scratch in PyTorch on a tiny transformer.
6. Read and write a TRL/PEFT recipe for the same pipeline on a Pythia-160M or SmolLM2-360M with LoRA on a single 24 GB GPU.

1 Motivation & intuition

1.1 Why classical RL is the wrong tool, naively

See the reading companion for full annotated citations.

The reader has spent seventeen weeks training agents on crisply-specified MDPs — Atari games where score changes supply reward, control suites where a sensor returns a number, grid worlds where a goal cell pays out +1. *Language is none of those things.* For an instruction-tuned model the "reward" is a fuzzy judgement ("was that a helpful response?"), the "state" is the entire prefix of text, the action space is the size of the vocabulary (often 30,000–200,000 tokens), and the trajectories last hundreds of steps. Worse, the reward is *not provided by the environment* at all: it lives inside a human reviewer's head.

The breakthrough of Ouyang et al. 2022[1] was to package this messy problem into a familiar shape. The reviewer's preferences over pairs of completions get distilled into a *neural reward model* $r_\phi(x, y)$; the language model is then a stochastic policy whose action is a token and whose state is the prefix. PPO — the same PPO that solved MuJoCo — becomes the workhorse.

Intuition. Think of RLHF as a two-step approximation. **Step one**: "what would a human say is better?" is hard, so we collect a few thousand pairwise judgements and *interpolate* them with a neural reward model. **Step two**: "what should the language model say?" is hard, so we run PPO against the interpolant, with a leash (a KL penalty) tying us back to the supervised baseline so we never drift too far into reward-hacking land.

1.2 The four-model picture

The defining ergonomic of classical RLHF is that *four* models sit in memory simultaneously (Figure 1.1). On a 24 GB consumer GPU this is uncomfortable even with LoRA — which is why the field eventually invented DPO and GRPO. We will see DPO next week.

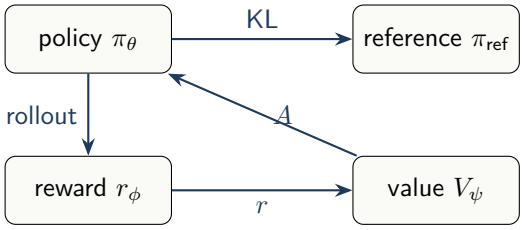


Figure 1.1: The four-model RLHF picture. The policy is trained; the reference is frozen at its SFT checkpoint; the reward model is frozen at the end of stage 2; the value head learns alongside the policy.

Common pitfall

A common confusion: "why do I need a critic if PPO has the clip?" The clip is a *trust region* on the policy update; it does not estimate advantages. Without a critic, advantages are just rewards minus a moving baseline — usable, but high-variance. Token-level PPO without a value head is what GRPO eventually re-introduces in week 20, with a clever

[1] InstructGPT, arXiv:2203.02155.

group-relative baseline replacing the critic.

2 Mathematical formulation

2.1 Stage 0: the supervised fine-tuning baseline

We begin with a base language model π_{base} (pre-trained on web text). Stage 1 of RLHF is *supervised fine-tuning* (SFT): maximise the log-likelihood of N curated demonstrations $\{(x_i, y_i)\}_{i=1}^{N}$,

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{|y_i|}\log \pi_\theta(y_{i,t}\,|\,x_i, y_{i,<t})\,. \tag{2.1}$$

After SFT the model is reasonably helpful but not deliberately *aligned*. We will denote the result π_{ref} — it plays a starring role in stages 2 and 3.

2.2 Stage 1: the Bradley–Terry reward model

Definition 2.2.1 (Bradley–Terry model, 1952)**.** For each prompt x and pair of candidate completions (y_w, y_l) ("winner" and "loser"), assume there exists a latent quality function $r^\star(x, y) \in \mathbb{R}$ such that the probability the annotator picks y_w over y_l is

$$\mathbb{P}(y_w \succ y_l\,|\,x) = \sigma(r^\star(x, y_w) - r^\star(x, y_l))\,, \quad \sigma(u) = \frac{1}{1+e^{-u}}. \tag{2.2}$$

The model is identifiable only up to an additive constant per prompt: shift $r^\star \mapsto r^\star + c(x)$ and (2.2) is unchanged.

Theorem 2.2.2 (Reward-model MLE)**.** *Given a preference dataset $\mathcal{D} = \{(x_i, y_w^{(i)}, y_l^{(i)})\}_{i=1}^{M}$ sampled i.i.d. from (2.2), the maximum-likelihood estimator of a parametric family r_ϕ minimises the cross-entropy loss*

$$\mathcal{L}_{RM}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]\,. \tag{2.3}$$

Proof. Each independent observation $\mathbf{1}\{y_w \succ y_l\}$ has Bernoulli likelihood $\sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$ by (2.2). The negative log-likelihood of the dataset is then exactly (2.3) up to a constant in ϕ. □ □

Remark. In practice r_ϕ is implemented as a transformer with a scalar linear head, applied to the concatenation $[x; y]$ and pooled at the last token (left-pad or attention-mask the prompt away from the loss). The two forward passes per pair, one for y_w and one for y_l, can be batched.

Common pitfall

The Bradley–Terry model assumes *transitive, deterministic* preferences from each annotator. Real annotators are noisy and internally inconsistent; Azar et al. 2024 (IPO) show that when preferences saturate ($\mathbb{P} \to 1$), the BT model pushes $\beta \to \infty$ in DPO. We discuss this fix in week 19.

2.3 Stage 2: the KL-regularised RL objective

Given a frozen reward model r_ϕ and frozen reference π_{ref}, the RLHF objective is to maximise

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\Big[\mathbb{E}_{y\sim\pi_\theta(\cdot\,|\,x)}[r_\phi(x,y)] \;-\; \beta\,\mathrm{KL}(\pi_\theta(\cdot\,|\,x)\,\|\,\pi_{\text{ref}}(\cdot\,|\,x))\Big]. \tag{2.4}$$

The KL term is a soft penalty on drift from the SFT baseline; it is not a hard upper bound on KL or a safety guarantee. Without it the policy will quickly find adversarial sequences that the reward model loves but that aren't even valid English — the canonical example being "aaaaaaaa…" for a sentiment model that loves repeated friendly characters.

Theorem 2.3.1 (Closed-form optimum of (2.4))**.** *For $\beta > 0$, fixed reward r, positive reference support on the considered responses, and finite $Z(x)$, the* unconstrained *maximiser of* (2.4) *over the simplex of policies $\pi(\cdot\,|\,x)$ is*

$$\pi^\star(y\,|\,x) = \frac{1}{Z(x)}\pi_{ref}(y\,|\,x)\,\exp\Big(\tfrac{1}{\beta}r(x,y)\Big), \qquad Z(x) = \sum_{y'}\pi_{ref}(y'\,|\,x)\,\exp\Big(\tfrac{1}{\beta}r(x,y')\Big). \tag{2.5}$$

Proof. Drop the prompt x to lighten notation. We maximise $\mathbb{E}_{y\sim\pi}[r(y)] - \beta\mathrm{KL}(\pi\|\pi_{\text{ref}})$ over π subject to $\sum_y \pi(y) = 1$. Form the Lagrangian

$$\mathcal{L}(\pi,\lambda) = \sum_y \pi(y)r(y) - \beta\sum_y \pi(y)\log\tfrac{\pi(y)}{\pi_{\text{ref}}(y)} - \lambda\Big(\sum_y \pi(y) - 1\Big).$$

Differentiating w.r.t. $\pi(y)$ and setting to zero,

$$r(y) - \beta\Big(\log\tfrac{\pi(y)}{\pi_{\text{ref}}(y)} + 1\Big) - \lambda = 0 \;\;\Longrightarrow\;\; \pi(y) = \pi_{\text{ref}}(y)\,\exp\Big(\tfrac{r(y)}{\beta} - 1 - \tfrac{\lambda}{\beta}\Big).$$

Normalising over y absorbs the constants $-1-\lambda/\beta$ into the partition function, giving (2.5). □ □

Remark. Equation (2.5) is the seed of *Direct Preference Optimization* (DPO). Inverting it gives $r(x,y) = \beta\log\frac{\pi^\star(y\,|\,x)}{\pi_{\text{ref}}(y\,|\,x)} + \beta\log Z(x)$, the *implicit reward* that we will use next week to bypass stages 2 and 3 entirely.

2.4 Stage 3: PPO at the token level

In practice we cannot enumerate $y \in \mathcal{A}^*$ to normalise (2.5). We instead *sample*, treating the language model as a Markov decision process whose state at step t is the prefix $s_t = (x, y_{<t})$ and whose action $a_t = y_t$ is a single token.

Definition 2.4.1 (Token-level RLHF MDP)**.** $\mathcal{S} = \mathcal{A}^*$ (all prefixes); $\mathcal{A}$ = the vocabulary; transitions are deterministic concatenation $s_{t+1} = (s_t, a_t)$; the per-step reward is

$$R_t = \begin{cases} -\beta(\log\pi_\theta(a_t\,|\,s_t) - \log\pi_{\text{ref}}(a_t\,|\,s_t)) & t < T, \\ -\beta(\log\pi_\theta(a_t\,|\,s_t) - \log\pi_{\text{ref}}(a_t\,|\,s_t)) + r_\phi(x, y_{1:T}) & t = T, \end{cases} \tag{2.6}$$

where T is the final-token index of the completion.

Remark (Why a per-token KL?). The expected sum of the per-step penalties equals $-\beta\mathrm{KL}(\pi_\theta(\cdot\,|\,x)\,\|\,\pi_{\text{ref}}(\cdot\,|\,x))$ by the chain rule of KL — so (2.6) is precisely the KL term of (2.4) unrolled across the autoregressive trajectory. Token-level credit assignment lets the value head and the advantages distinguish "it was the third word that went wrong" from "the whole thing was bad", dramatically reducing variance.

Definition 2.4.2 (Generalised Advantage Estimation, Schulman et al. 2016)**.** For a learned value estimator V_ψ and a rollout $(s_0, a_0, R_0, \ldots, s_T, a_T, R_T)$, define $\delta_t = R_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$. The GAE-$\lambda$ advantage is

$$\hat{A}_t = \sum_{k=0}^{T-t} (\gamma\lambda)^k \delta_{t+k}. \tag{2.7}$$

In language settings the natural discount is $\gamma = 1$ (an episode is one sequence, and discounting would change the relative value of early and late token rewards; different prefixes are different states), and $\lambda \in [0.9, 0.97]$ is standard. We bootstrap $V_\psi(s_{T+1}) = 0$.

Definition 2.4.3 (PPO clipped surrogate)**.** With $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ the per-token importance ratio and $\hat{A}_t$ a GAE estimate,

$$\mathcal{L}^{\text{PPO}}(\theta) = -\mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon)\,\hat{A}_t\Big)\Big]. \tag{2.8}$$

The clip ε is typically 0.2.

Theorem 2.4.4 (Why the clip is a pessimistic surrogate)**.** *For every sample with $\hat{A}_t \geq 0$, $\min(r_t\hat{A}_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t)$ is a* lower bound *on $r_t\hat{A}_t$. For every sample with $\hat{A}_t < 0$ it is again a lower bound. Consequently* (2.8) *upper-bounds the negative unclipped surrogate loss. The clipped sample contributes no gradient incentive for further movement in its favourable direction; shared parameters, other samples, and other loss terms can still move its ratio farther.*

Proof. Case $\hat{A}_t \geq 0$. If $r_t > 1+\varepsilon$, the clip gives $(1+\varepsilon)\hat{A}_t \leq r_t\hat{A}_t$; the min picks the clipped value. Gradient w.r.t. θ through the clipped branch is zero in r_t, so increasing r_t further does *not* decrease the loss. Case $\hat{A}_t < 0$ is symmetric: if $r_t < 1-\varepsilon$, the clip $(1-\varepsilon)\hat{A}_t$ is again the min, gradient is zero, so this sample alone provides no gradient incentive to push r_t farther down. □ □

Intuition. The intuition behind the clip is *trust region by tape.* A proper trust region (TRPO) constrains $\text{KL}(\pi_{\text{old}} \| \pi_\theta) \leq \delta$ via a second-order line search; PPO replaces that constraint with a min / max tape on the per-sample ratio. It is much cheaper and almost as well-behaved.

2.5 The full loss

Combining the policy loss (2.8), the value loss, an entropy bonus, and the token-level KL penalty already inside the rewards,

$$\mathcal{L}_{\text{total}}(\theta, \psi) = \mathcal{L}^{\text{PPO}}(\theta) + c_v \cdot \mathbb{E}_t\Big[(V_\psi(s_t) - G_t)^2\Big] - c_e \cdot \mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))]. \tag{2.9}$$

Here $G_t = \hat{A}_t + V_\psi(s_t)$ is the return target. Coefficients $c_v \approx 0.5$, $c_e \in [0, 0.01]$.

2.6 The adaptive KL controller

The InstructGPT recipe lets β float to keep the observed KL around a target $\text{KL}^{\text{target}}$ (typically 0.05–0.2). The Schulman 2017 update rule is

$$\beta \leftarrow \beta \cdot \left(1 + \text{clip}\left(\frac{\text{KL}^{\text{obs}}}{\text{KL}^{\text{target}}} - 1,\ -0.2,\ +0.2\right) \cdot \frac{1}{H}\right), \tag{2.10}$$

with horizon $H \approx 10$ rollouts. When KL drifts above target, β rises and pulls the policy back; when KL undershoots, β relaxes and lets the policy explore.

Common pitfall

With small batch sizes the observed KL is noisy and (2.10) oscillates. Two fixes: (i) accumulate KL across several rollouts before updating β; (ii) cap β in a hard range $[\beta_{\min}, \beta_{\max}]$.

2.7 A picture of token-level credit assignment



Figure 2.1: Token-level reward shaping. Each response token earns a signed reward adjustment proportional to its sampled log-ratio; the final token additionally receives the scalar reward-model score.

2.8 An empirical illustration of the BT loss

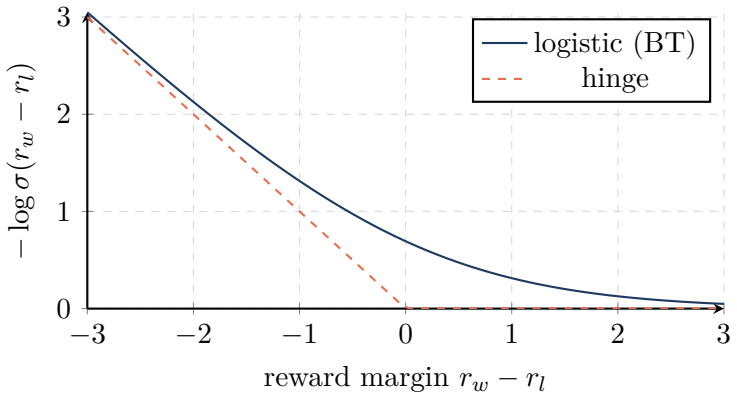


Figure 2.2: The Bradley–Terry logistic loss versus the hinge variant used by Slic-HF. The logistic never saturates to zero, which is both a strength (always provides gradient) and a weakness (saturated preferences push β to infinity in DPO, motivating IPO).

3 Algorithms

3.1 The three-stage pipeline

Algorithm 1: Three-stage RLHF (InstructGPT-style).

Input: base model π_{base}; demos $\mathcal{D}_{\text{SFT}}$; preferences $\mathcal{D}_{\text{pref}}$; prompts $\mathcal{D}_{\text{prompt}}$
Output: aligned policy π_θ
Stage 1 (SFT): initialise $\pi_\theta \leftarrow \pi_{\text{base}}$
for *epoch* $= 1, \ldots, E_{SFT}$ **do**
  minimise $\mathcal{L}_{\text{SFT}}$ (2.1) on a batch of $\mathcal{D}_{\text{SFT}}$
end
$\pi_{\text{ref}} \leftarrow \pi_\theta$ `// freeze`
Stage 2 (RM): initialise r_ϕ from π_{ref} + scalar head
for *epoch* $= 1, \ldots, E_{RM}$ **do**
  minimise $\mathcal{L}_{\text{RM}}$ (2.3) on a batch of $\mathcal{D}_{\text{pref}}$
end
Stage 3 (PPO): initialise value head V_ψ; KL coefficient β
for *step* $= 1, \ldots, S$ **do**
  sample a batch of prompts X from $\mathcal{D}_{\text{prompt}}$
  for each prompt, sample a completion under π_θ
  compute per-token rewards (2.6) using $\pi_{\text{ref}}, r_\phi, \beta$
  compute GAE advantages (2.7) from V_ψ
  for *epoch* $= 1, \ldots, K$ **do**
    update (θ, ψ) by Adam on $\mathcal{L}_{\text{total}}$ (2.9)
  end
  update β via (2.10)
end

Complexity. Per PPO step, we run one rollout (one forward pass per token in the completion) plus K training epochs each requiring one forward+backward over the rollout. For batch B, completion length L, K=4 epochs the cost is $\mathcal{O}(B \cdot L)$ rollout-time + $\mathcal{O}(K \cdot B \cdot (P + L))$ training-time, where P is the prompt length. Memory is dominated by the four models; LoRA shrinks the policy and value-head footprint by 100× vs. full FT.

3.2 Worked numerical example

Consider a toy preference batch of two items:

$$(x_1,\, y_w^{(1)},\, y_l^{(1)}) = (\text{“cats”},\, \text{“i like cats.”},\, \text{“CATS ARE BAD!!”}),$$
$$(x_2,\, y_w^{(2)},\, y_l^{(2)}) = (\text{“tea”},\, \text{“yes, tea is nice.”},\, \text{“NO!!”}).$$

Suppose after a few RM steps the network outputs $r_\phi(x_1, y_w) = 0.8$, $r_\phi(x_1, y_l) = -0.2$, $r_\phi(x_2, y_w) = 0.5$, $r_\phi(x_2, y_l) = 0.1$. Then the BT loss (2.3) evaluates to

$$-\tfrac{1}{2}[\log \sigma(0.8 - (-0.2)) + \log \sigma(0.5 - 0.1)] = -\tfrac{1}{2}[\log \sigma(1.0) + \log \sigma(0.4)] \approx 0.428.$$

Pairwise accuracy is 1.0 (both differences positive). After another epoch, the gaps widen to $(1.6, 0.9)$ and the loss falls to ≈ 0.291. Compare the empirical curves in Figure 5.1.

4 Implementation walkthrough

The full project lives in project/; the highlights:

- `project/model.py` — a 2-layer 64-dim causal transformer that doubles as policy backbone and reward-model backbone. `ValueHead` and `RewardModel` bolt scalar heads onto the trunk.
- `project/sft.py` — a 30-line SFT loop.
- `project/rm.py` — Bradley–Terry training with `reward_loss` implementing (2.3).
- `project/ppo.py` — token-level rollouts, GAE, clipped surrogate, adaptive KL controller.
- `project/trl_recipe.py` — the same pipeline expressed as a TRL/PEFT script for a real LM (SmolLM2-360M).

Reward-model loss (annotated).

```
def reward_loss(r_w, r_l):
    # Bradley--Terry cross-entropy.  No reduction inside the sigmoid
    # because we want -log P(winner > loser) per pair.
    return -F.logsigmoid(r_w - r_l).mean()
```

Per-token KL-shaped rewards (annotated).

```
# kl_token = log pi_theta(a_t | s_t) - log pi_ref(a_t | s_t)
kl_token = logprobs - ref_logprobs
per_tok_rewards = -kl_coef * kl_token
# Sparse RM reward only at the final position.
per_tok_rewards[:, -1] += rm_scores
```

GAE (annotated).

```
def compute_gae(rewards, values, gamma=1.0, lam=0.95):
    B, T = rewards.shape
    adv = torch.zeros_like(rewards)
    last = torch.zeros(B, device=rewards.device)
    for t in reversed(range(T)):
        delta = rewards[:, t] + gamma * values[:, t+1] - values[:, t]
        last  = delta + gamma * lam * last
        adv[:, t] = last
    returns = adv + values[:, :T]
    return adv, returns
```

PPO step (annotated).

```
ratio = (new_logprobs - rollout.logprobs).exp()
unclipped = ratio * adv
```

```
clipped   = torch.clamp(ratio, 1-clip_eps, 1+clip_eps) * adv
policy_loss = -torch.min(unclipped, clipped).mean()
value_loss  = F.mse_loss(v_pred, returns)
loss = policy_loss + vf_coef * value_loss - ent_coef * entropy
```

TRL on a real LM (24 GB GPU). For the same recipe on Pythia-160M or SmolLM2-360M, the production code is much shorter because TRL wraps everything; see `project/trl_recipe.py` for the SFT, RM, and PPOv2 configurations. Recommended LoRA: r=16, α=32, on `["q_proj","k_proj","v_proj","o_proj"]`; batch 4; grad-accum 8; lr 5×10^{-5} (SFT), 1×10^{-4} (RM LoRA), 1×10^{-6} (PPO); $\beta = 0.05$ adaptive; expect $\sim$ 1k prompts/hour. The TRL recipe is dossier-sourced (cf. Lambert et al. 2024, *Tülu 3*, arXiv:2411.15124).

5 Empirical results

Figure 5.1 shows the supervised pre-training and reward-model training curves on the toy dataset; the RM reaches close to 100% pairwise accuracy in 150 steps in well under five seconds on CPU. Figure 5.2 shows the PPO reward curve: starting from ≈ 0, the average reward-model score climbs above 4 in 30 PPO updates while the adaptive KL controller keeps the empirical KL near 0.05.

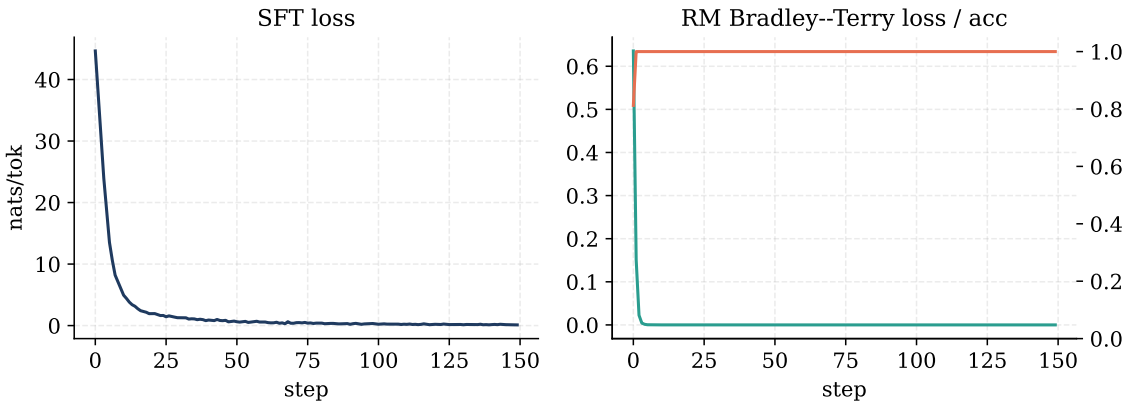


Figure 5.1: Stage 1 (SFT) and stage 2 (reward model) training curves on the toy preference dataset. Generated by `project/make_figures.py`, seed 0.

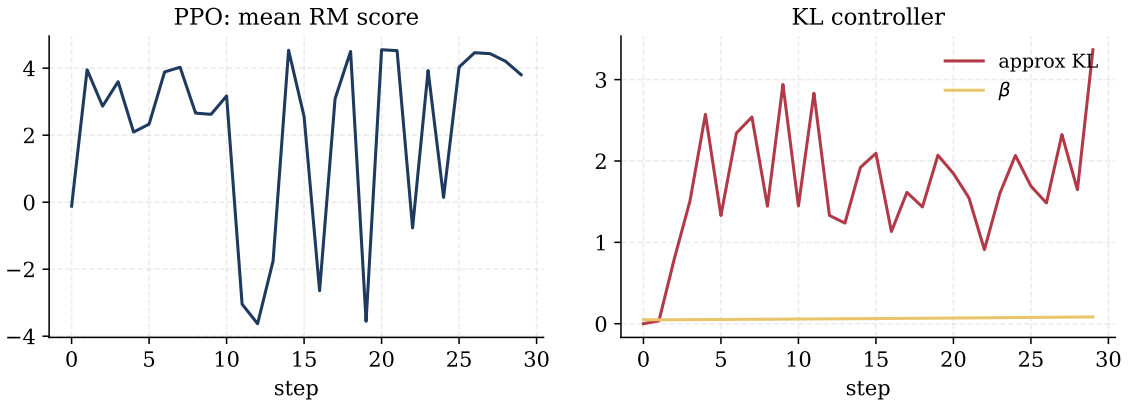


Figure 5.2: Stage 3 (PPO) reward and KL trajectories. The adaptive controller raises β when KL spikes and relaxes it when KL is under target. Generated by `project/make_figures.py`, seed 0.

Figure 5.3 is a schematic of reward-model overoptimisation in the spirit of Gao, Schulman, Hilton 2023: at small KL, proxy and gold reward agree; as KL grows, the policy "hacks" the proxy and gold reward falls.

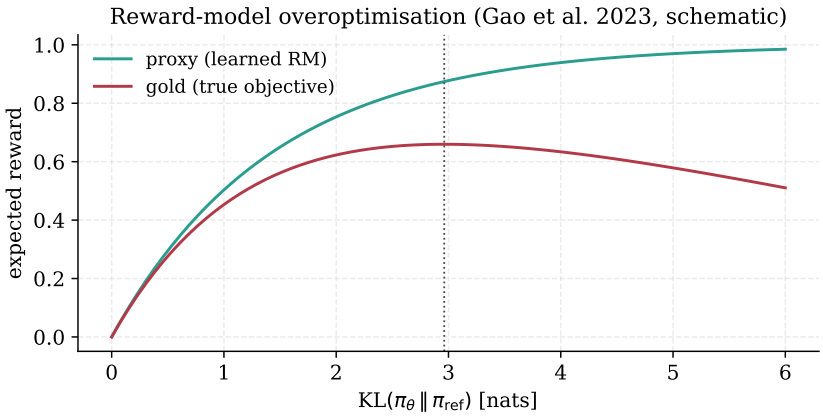


Figure 5.3: Reward-model overoptimisation (schematic). The gold reward peaks at a finite KL and then falls; the proxy keeps rising.

6 Theoretical deepening

6.1 Policy-gradient identity for autoregressive LMs

We started with the surrogate (2.8); let us anchor it in the underlying policy-gradient identity. For a stochastic policy $\pi_\theta(a \mid s)$ and an episodic return G,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\right]$$

where $\hat{A}_t$ is any baseline-corrected estimator of the advantage. Importance-sampling onto a slightly older policy $\pi_{\theta_{\text{old}}}$, the same gradient is the gradient of $\mathbb{E}_t[r_t(\theta)\hat{A}_t]$ at $\theta = \theta_{\text{old}}$. PPO's contribution is that the clipped *lower bound* on $r_t(\theta)\hat{A}_t$ has the same gradient at the on-policy point *and* prevents the optimiser from straying too far in one update.

Lemma 6.1.1 (Local gradient equivalence at $\theta = \theta_{\text{old}}$)**.** *At $r_t = 1$, $\nabla_\theta r_t(\theta)\hat{A}_t = \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$, equal to the policy-gradient estimator term. So PPO is unbiased at the on-policy point; the clip can matter once r_t leaves $[1-\varepsilon, 1+\varepsilon]$, depending on the sign of $\hat{A}_t$. Unbiasedness also requires an appropriate advantage estimator.*

Remark. At the first update on fresh on-policy data, ratios equal one irrespective of batch size. Reusing that batch after parameter updates makes it off-policy. Batch size affects sampling noise; learning rate, update count, and parameter sharing determine how far ratios move. Clipping is not a hard trust region.

6.2 Why a value head reduces variance

Without a critic the natural advantage estimator on a token-level RLHF trajectory is the *tail* of the per-token rewards: $\hat{A}_t = \sum_{k \geq t} R_k - b(s_t)$ for some scalar baseline b. The variance of this estimator is dominated by the variance of the final-token RM reward, which is *O(1)* regardless of t — so the per-token gradients all carry full-trajectory variance. GAE with a learned V_ψ cuts this variance by replacing the tail sum with $V_\psi(s_{t+1})$ wherever $\lambda < 1$:

$$\hat{A}_t = \delta_t + \gamma\lambda\delta_{t+1} + \cdots, \quad \delta_t = R_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t). \tag{6.1}$$

At $\lambda = 0$ the advantage is the one-step TD error, which is still random across sampled actions, stochastic rewards, or transitions even with a perfect V_ψ; an approximate critic can also introduce bias. At $\lambda = 1$ the advantage is the unbiased Monte-Carlo return (minus the baseline), with full variance. The sweet spot for RLHF is usually $\lambda \in [0.9, 0.97]$.

Intuition. GAE is the bias-variance dial. λ buys variance reduction at the cost of bias. The variance you remove is precisely the part of the return that should have been credited to *later* states than the current one.

6.3 Equivalence between the per-token KL and the sequence KL

We claimed in (2.6) that the sum of per-token KL penalties equals the sequence-level KL. Let us prove it.

Theorem 6.3.1 (Chain rule of KL for autoregressive policies)**.** *For any two policies π, μ on the same vocabulary and any prompt x,*

$$\mathrm{KL}(\pi(\cdot \mid x) \,\|\, \mu(\cdot \mid x)) = \mathbb{E}_{y\sim\pi}\left[\sum_{t=1}^{|y|} \log \frac{\pi(y_t \mid x, y_{<t})}{\mu(y_t \mid x, y_{<t})}\right].$$

Proof. By definition,

$$\mathrm{KL}(\pi\|\mu) = \mathbb{E}_{y\sim\pi}\left[\log \frac{\pi(y \mid x)}{\mu(y \mid x)}\right].$$

Factor each conditional via the chain rule: $\pi(y \mid x) = \prod_t \pi(y_t \mid x, y_{<t})$, and similarly for μ. The logarithm turns the product ratio into a sum that matches the right-hand side. □ □

This identity is what makes "per-token KL penalty" algorithmically honest: the expectation of the sampled log-ratio sum equals the sequence KL. One sampled sum can be negative; the KL itself cannot.

6.4 The KL controller as a PI loop in log-space

The adaptive controller (2.10) is a stripped-down proportional controller acting on $\log \beta$. Let $e = \mathrm{KL}^{\mathrm{obs}}/\mathrm{KL}^{\mathrm{target}} - 1$ be the error signal. Then $\beta_{n+1} = \beta_n(1 + \mathrm{clip}(e, \pm 0.2)/H)$, i.e. $\log \beta_{n+1} \approx \log \beta_n + \mathrm{clip}(e, \pm 0.2)/H$. This is a P-controller in log-space with horizon H. In production recipes (Tülu-3, Open-Reasoner-Zero), the clip is often dropped *entirely* for verifiable-reward setups — the implicit regulariser is the verifier itself, not a leash on the SFT.

6.5 Common questions, answered

Why does RLHF work at all? Because the implicit reward surface near the SFT policy is dense and well-shaped: helpfulness, harmlessness, tone — these are all smooth functions of the response that the BT loss models well within a small KL ball around the SFT.

Why does PPO "do better than DPO" in some papers? Three reasons: (i) PPO sees on-policy data, fixing the distribution-shift problem of offline DPO; (ii) the value head soaks up token-level variance; (iii) the KL leash is adaptive, not fixed. None of these is fundamental; modern iterative DPO closes all three gaps.

Why does PPO need a frozen reference but DPO does not (operationally)? Because DPO's reference is *baked into the implicit reward parametrisation*, not used as a runtime entropy regulariser. Both methods carry the reference; PPO writes it in a separate set of weights, DPO writes it in the formula.

Can I skip stage 1 (SFT)? You can, and people do — "RL from base" is the headline of DeepSeek-R1-Zero. But on open-ended preference data the base model's distribution is so flat that the KL leash has nowhere to anchor, and reward hacking is wildly easier.

Is the BT model identifiable? Only up to an additive per-prompt constant. This is fine for ranking but means you cannot compare scores across prompts.

6.6 A second worked numerical example: one PPO step end-to-end

Suppose at step 17 of PPO, the rollout for one prompt has yielded a 6-token completion with per-token log-probabilities $\log \pi_\theta = (-1.0, -1.2, -0.8, -1.5, -2.0, -1.0)$, $\log \pi_{\mathrm{ref}} = (-1.1, -1.3, -0.7, -1.4, -1.8, -1.1)$. The per-token KL increments are

$(0.1, 0.1, -0.1, -0.1, -0.2, 0.1)$, summing to -0.1 — the policy assigns this particular response lower probability than the reference. This negative sample is not a negative KL and cannot establish whether a batch-level KL target is met.

The reward model assigns the final completion $r_\phi = +0.8$. With $\beta = 0.05$, the per-token rewards are $(-0.005, -0.005, +0.005, +0.005, +0.010, +0.795)$ — nearly all the signal is at the final token, as expected.

The value head currently predicts $V = (0.0, 0.0, 0.0, 0.0, 0.0, 0.4)$ (it has learned that good rollouts end with ~ 0.8 reward but hasn't propagated credit backward yet). With $\gamma = 1, \lambda = 0.95$, the GAE TD errors are $\delta = (-0.005, -0.005, 0.005, 0.005, 0.410, 0.395)$. Starting from $\hat{A}_6 = 0.395$ and using $\hat{A}_t = \delta_t + 0.95\hat{A}_{t+1}$ gives the raw advantages $\hat{A} \approx (0.63864, 0.67752, 0.71844, 0.75099, 0.78525, 0.39500)$. Their return targets are $\hat{A} + V$; the final target is 0.795. Batch normalisation, if used, requires the full batch and cannot be inferred from this single response. For token 5, a new-versus-old log-probability increase of 0.01 gives $\rho = e^{0.01} \approx 1.01005$, so its unclipped contribution is $1.01005 \cdot 0.78525 \approx 0.79314$. With $\varepsilon = 0.2$ it is not clipped. An Adam learning rate does not bound the resulting log-probability change; inspect the actual ratios and KL after the update.

7 Safety angle

Safety angle

RLHF *is* the alignment training pipeline shipped in every major production language model from 2022 to 2024. Its known failure modes are not academic curiosities; they are recurring incidents.

Reward-model overoptimisation. Gao, Schulman, Hilton 2023 show analytically and empirically that the proxy-vs-gold gap grows roughly with $\sqrt{\mathrm{KL}}$ on simple-RM scaling laws; gold reward peaks at a finite KL and then declines (Figure 5.3). *Mitigation*: early-stopping by gold-eval on held-out human preferences; KL penalty; reward-model ensembles; iterative refresh of preference data.

Length bias. Longer outputs receive larger absolute per-token KL costs but also larger absolute reward; PPO with mean per-token loss tilts toward verbosity (the same mechanism SimPO and Dr.GRPO address explicitly). *Mitigation*: length normalisation in the reward; or token-level (sum) loss as in DAPO/Dr.GRPO; or an explicit per-prompt length cap.

Reward hacking via likelihood displacement. The policy can win at the proxy by raising the probability of a small set of high-reward strings sharing surface features with the training set — the canonical "model that ends every answer with 'I hope this helps!"' — without improving on the underlying task. *Mitigation*: KL leash; regular human spot-checks; preference-data refresh; iterative DPO (see week 19, §iterative).

Catastrophic Goodhart. arXiv:2407.14503 (2024) prove that under heavy-tailed reward misspecification, *no* finite KL bound suffices. The rigorous answer is to reduce misspecification (process reward models, verifiable rewards), not to crank β.

Sycophancy and refusal collapse. Two empirically common failure modes that the InstructGPT paper itself warns about: preference data is biased toward agreement, and PPO will obediently amplify that. *Mitigation*: explicit factuality datasets; red-teaming prompts; rule-based corrections during data collection.
Spend a full afternoon reading Gao et al. 2023 before shipping anything RLHF-trained to production. The scaling laws are small, simple, and uncomfortably accurate.

8 Matching application

Matching application

The lead-routing problem at synthetic insurance is, formally, a contextual bandit: a lead x arrives, the system picks a broker a, and we observe a delayed conversion signal r. *RLHF in this setting* reframes "which broker did the lead pick when shown two options?" as a Bradley–Terry pairwise preference dataset. We can then train a reward model $r_\phi(x, a)$ on those clicks, freeze it, and use PPO (or a contextual-bandit reduction thereof) to nudge a learned router toward broker assignments that historic leads would have preferred, while a KL penalty against the live rule-based router keeps us inside the regulatory comfort zone. The same template applies to synthetic ATS: candidates' apply-or-skip decisions on a small randomised set of job suggestions form pairwise preferences that we can distil into a ranking reward, with a KL leash to the production keyword-match baseline. Two warnings carry over directly: (i) the *reward-model overoptimisation* caveat applies in full force — the proxy here is broker-click probability, the gold is broker–customer six-month retention — and the gap between them is exactly what cost the business £40k in 2025 Q3; (ii) *length bias* translates into *result-set bias*: PPO will happily learn to surface more candidates per query to bump the click-through reward, drowning recruiters in volume.

9 Exercises

The graded version (with hidden tests) lives in `workbook.ipynb`. Short summary:

1. ($\star$) Implement `reward_loss(r_w, r_l)` returning the Bradley–Terry cross-entropy.

   Intuition. Use `F.logsigmoid`, not `torch.log(torch.sigmoid(...))`; the latter is numerically unstable for large $|r_w - r_l|$.

2. ($\star$) Compute the per-position KL between two logit tensors.

3. ($\star\star$) Implement the PPO clipped surrogate from scratch.

4. ($\star\star$) Implement GAE from scratch.

5. ($\star\star$) Implement the adaptive KL controller update rule.

6. ($\star\star$) Add a length-normalised reward modifier; show it reduces verbosity in the generated samples.

7. ($\star\star\star$) Compute the unnormalised log-probability of the closed-form RLHF optimum — this is the stepping stone to week 19's DPO loss.

8. ($\star\star\star$) *Open-ended:* build a "proxy-vs-gold" diagnostic that flags overoptimisation when the learned reward and an oracle heuristic diverge.

9. ($\star\star$) *Meaningful objective:* measure the correlation between the heuristic reward and the learned RM scores on a fresh batch of samples; report it before and after PPO to detect drift.

10 Notation, conventions, and detailed pseudocode

10.1 Notation cheat sheet

Symbol	Meaning
x	prompt (token sequence)
y	response (token sequence)
s_t	RL state $= (x, y_{<t})$
a_t	RL action $= y_t$, a single token
T	length of the response (last-token index)
$\pi_\theta(a \mid s)$	policy probability
π_{ref}	frozen reference policy (= SFT checkpoint)
$r_\phi(x, y)$	learned reward-model score (scalar, sequence-level)
$V_\psi(s)$	value head (scalar, per-token)
R_t	per-step reward (token-level, per (2.6))
δ_t	TD error $R_t + \gamma V(s_{t+1}) - V(s_t)$
$\hat{A}_t$	GAE advantage
$r_t(\theta)$	importance ratio $\pi_\theta / \pi_{\theta_{\text{old}}}$
β	KL coefficient
ε	PPO clip range (typ. 0.2)
γ	RL discount (1.0 for LLMs)
λ	GAE bias-variance dial
$\mathcal{H}$	token-level entropy

10.2 Detailed PPO rollout pseudocode

Algorithm 2: One PPO rollout batch with GAE.

Input: policy π_θ; reference π_{ref}; reward r_ϕ; value V_ψ; prompts X; max-new-tokens L; β, γ, λ

Output: advantages $\hat{A}$, returns G, rollout caches

for x *in* X *(in parallel)* **do**
- $s_0 \leftarrow x$
- **for** $t = 1, \ldots, L$ **do**
  - $a_t \sim \pi_\theta(\cdot \mid s_{t-1})$ `// ancestral sample`
  - record $\log \pi_\theta(a_t \mid s_{t-1})$, $\log \pi_{\text{ref}}(a_t \mid s_{t-1})$, $V_\psi(s_{t-1})$
  - $s_t \leftarrow (s_{t-1}, a_t)$
- **end**
- $r \leftarrow r_\phi(x, y_{1:L})$ `// single RM scalar for the rollout`
- **for** $t = 1, \ldots, L$ **do**
  - $R_t \leftarrow -\beta \left(\log \pi_\theta(a_t \mid s_{t-1}) - \log \pi_{\text{ref}}(a_t \mid s_{t-1})\right)$
- **end**
- $R_L \leftarrow R_L + r$ `// add sparse RM reward at the final step`
- `// GAE backward sweep`
- $V_{L+1} \leftarrow 0$; $\hat{A}_{L+1} \leftarrow 0$
- **for** $t = L, L-1, \ldots, 1$ **do**
  - $\delta_t \leftarrow R_t + \gamma V_\psi(s_t) - V_\psi(s_{t-1})$
  - $\hat{A}_t \leftarrow \delta_t + \gamma\lambda \hat{A}_{t+1}$
- **end**
- $G_t \leftarrow \hat{A}_t + V_\psi(s_{t-1})$ `// return target`

end

return all caches

10.3 Detailed PPO update pseudocode

Algorithm 3: Multi-epoch PPO update.

Input: rollout caches; ε, c_v, c_e, K, lr

for *epoch* $= 1, \ldots, K$ **do**
- for every position (t, b) in the batch:
  - $\log \pi_{\text{new}}(a_t \mid s_{t-1})$ from a fresh fwd pass
  - $r_t \leftarrow \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$
  - $L_t^{\text{p}} \leftarrow -\min(r_t \hat{A}_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t)$
  - $L_t^{\text{v}} \leftarrow (V_\psi(s_{t-1}) - G_t)^2$
  - $\mathcal{H}_t \leftarrow$ entropy of $\pi_\theta(\cdot \mid s_{t-1})$
- $\mathcal{L} \leftarrow \text{mean}(L^{\text{p}}) + c_v\,\text{mean}(L^{\text{v}}) - c_e\,\text{mean}(\mathcal{H})$
- $\theta \leftarrow \theta - \text{lr} \cdot \nabla_\theta \mathcal{L}$ `// Adam, grad-clipped`

end

10.4 Memory accounting on a 24 GB GPU

For Pythia-160M in bf16 with LoRA $r = 16$ on QKVO, the per-component peak VRAM is approximately:

Component	VRAM (GB)
Policy π_θ (bf16 + LoRA grads/state)	1.2
Reference π_{ref} (bf16, no grad)	0.4
Reward r_ϕ (bf16, no grad)	0.4
Value V_ψ (a scalar head sharing the trunk)	0.05
Rollouts (batch $\times$ seq) + activation memory	1.5
Adam state + LoRA grads	0.5
Total (with headroom)	≈ 4.5

A SmolLM2-360M scale-up roughly doubles the policy and reference columns; the total stays comfortably under 12 GB. Above 1.5B parameters, 4-bit (QLoRA) or sequence chunking become necessary.

10.5 Hyper-parameter recipe summary

Stage	Hyper-parameter	Recommended
SFT	lr	5×10^{-5} (full FT), 1×10^{-4} (LoRA)
SFT	batch	4 with grad-accum 4
SFT	epochs	1–3 over the demo set
RM	lr	1×10^{-4} LoRA
RM	batch	4–8 pairs per device
RM	target pairwise acc	> 0.7 on held-out
PPO	lr	1×10^{-6}
PPO	batch	4 prompts, grad-accum 8
PPO	rollout length	128–256 tokens
PPO	PPO epochs K	4
PPO	clip ε	0.2
PPO	γ, λ	1.0, 0.95
PPO	init β	0.05; target KL 0.1; adaptive
PPO	total steps	500–2000

11 Historical context and the road to GRPO

11.1 From REINFORCE to InstructGPT, in five papers

1. **REINFORCE** (Williams 1992). The score-function estimator $\nabla J = \mathbb{E}[\nabla \log \pi \cdot G]$. The mother of all policy-gradient methods. RLHF's gradient is REINFORCE under the hood.

2. **TRPO** (Schulman et al. 2015). Adds a hard KL trust-region constraint. Solves the policy-gradient instability problem but requires second-order methods.

3. **PPO** (Schulman et al. 2017). Replaces the trust region with a per-sample clipped ratio. First-order, works out of the box, becomes the default within a year.

4. **GPT-2 reward-model fine-tuning** (Ziegler et al. 2019). The first paper to apply PPO to a language model with a learned reward. Pioneer of the entire modern RLHF lineage.

5. **InstructGPT** (Ouyang et al. 2022). The industrial recipe: SFT $\to$ RM $\to$ PPO at scale, with adaptive KL. Ships in production as ChatGPT in November 2022.

11.2 Why GRPO replaced PPO for reasoning RL

PPO's value head buys variance reduction at the cost of a fourth model in memory. For *reasoning* tasks — where the reward is a sparse, often binary verifier signal at the end of a long chain of thought — the value head struggles to fit, and the rollout budget is dominated by long generations. Shao et al. (DeepSeekMath, 2024) realised that if you sample G rollouts per prompt and use their group-relative reward as the baseline, you can drop the critic entirely. This is GRPO; the rest of the algorithm (clip, KL leash) is PPO unchanged. Week 20 is devoted to it.

GRPO's relationship to PPO is analogous to A2C's relationship to DQN: an architectural simplification that becomes practical only once compute allows you to amortise the missing variance reduction through bigger batches. On a 24 GB GPU, $G = 8$ rollouts per prompt is comfortable; on a multi-GPU cluster, $G \geq 32$ is typical.

11.3 What was wrong with classical RLHF, exactly?

By 2024, three things had become clear:

1. The reward model is the weakest link. RM scaling laws (Gao et al. 2023) say the proxy-vs-gold gap grows monotonically with KL; modern PRMs and GenRMs attack this.

2. PPO is over-engineered for the LLM setting. The clip, GAE, value head, adaptive KL — this is a lot of moving parts relative to the underlying "maximise BT-likelihood with a KL leash" goal. DPO showed you can drop almost all of it.

3. Length and refusal biases are systemic. Both stem from misaligned per-token loss accounting; SimPO's length normalisation and Dr.GRPO's sum-not-mean accounting are the cleanest fixes.

The week 19 zoo and the week 20 GRPO family are the field's response to these three observations.

12 Further reading

See `extended_reading.pdf` for annotated summaries of Ouyang 2022 (InstructGPT), Schulman 2017 (PPO), Gao, Schulman, Hilton 2023 (scaling laws for reward overoptimisation), Bai 2022 (Constitutional AI / RLAIF), and Lambert 2024 (Tülu 3). Sourced from `research/llm_rl_sota.md` §1, §5, §6, §7.